Usuwanie tokenów pośrednich w treningu BPE

Michał Kędrzyński · 5 listopada 2026 · notatka z eksperymentu w repozytorium `ailab`

Wnioski. Sprawdziliśmy prosty wariant treningu BPE, który usuwa ze słownika tokeny pochłonięte przez dłuższe scalenia i oddaje ich miejsca nowym tokenom. Przy tym samym rozmiarze słownika fertility na teście spada na próbce z 26 źródeł Polish DynaWord o **–0,86%** przy 4534 tokenach i o **–0,35%** przy 30 000, poprawiając się na wszystkich 3 podziałach we wszystkich 40 punktach pomiaru. Efekt jest mały, ale powtarzalny. Prosta reguła progu 50% wypada praktycznie tak samo jak reguła wyprowadzona z bilansu kosztów, choć wykonuje około 2,5 raza więcej usunięć. Podobne podejścia są już opisane w literaturze, więc nie jest to nowa metoda. Wynik dotyczy fertility: nie mierzyliśmy jeszcze jakości modelu.

1. Problem

BPE dokłada token przy każdym scaleniu, ale nie usuwa tokenów, które stały się zbędne. Część z nich pełni tylko rolę etapów pośrednich. Przykład: korpus `abcd`×100, `ef`×30, `gh`×20, `ij`×10 i budżet trzech nowych tokenów. Klasyczne BPE daje `cd`, `bcd`, `abcd`, czyli 220 tokenów w wyniku, a dwa z nowych tokenów nie pojawiają się w nim wcale. Słownik `abcd`, `ef`, `gh` daje 170 tokenów, o 23% mniej. Koszt widać na nowym tekście: `cde` ma 2 tokeny w pierwszym wariancie i 3 w drugim. Dlatego osobno mierzymy słowa niewidziane.

2. Metoda

Bieżąca liczność. Po scaleniu pary odejmujemy liczbę scaleń od liczności obu rodziców. L to więc dokładna liczba wystąpień tokenu w aktualnym stanie korpusu, a n_0 to jego liczność przy wejściu do słownika. Sprawdzamy tylko rodziców ostatniego scalenia, bo tylko ich L maleje. Znaki bazowe nigdy nie są usuwane, więc każdy tekst da się zapisać.

Reguły. Porównujemy dwa warianty:

- **reguła 50%:** usuń token, gdy $L < n_0/2$;
- **reguła $L < f_{\text{best}}$:** usuń token, gdy koszt $L\cdot(k-1)$ jest mniejszy od zysku f_{best}, gdzie k to liczba części po rozkładzie, a f_{best} to liczność najczęstszej pary. Każda taka zamiana ściśle zmniejsza łączną liczbę tokenów.

Związek między regułami. Dla $k = 2$ i bez nakładek typu `aaa` L maleje tylko przy scaleniu, które zabiera m wystąpień, a f_{best} w klasycznym BPE nie rośnie. Opłacalność przy pierwszym usunięciu oznacza $L' < f_{\text{next}} \le m$, czyli $L' < L/2 \le n_0/2$. Próg 50% jest więc warunkiem koniecznym opłacalności, ale nie wystarczającym. Sprawdziliśmy to symulacją na klasycznym BPE: na 1500 losowych korpusach bez nakładek było 987 616 opłacalnych stanów tokenów i 0 wyjątków, oraz 106 045 stanów poniżej progu, ale nieopłacalnych. Na korpusach z nakładkami wyjątki stanowiły około 0,4–0,8%.

Samonaprawa. Błędne usunięcie, czyli takie przy $L > f_{\text{best}}$, zwykle cofa się samo: para rodziców staje się najczęstsza i w kolejnej rundzie wraca ten sam token. Tłumaczy to podobieństwo wyników obu reguł. Reguła 50% płaci głównie dodatkowymi rundami.

Rozmiar słownika i kodowanie. Trening kończy się przy docelowej liczbie tokenów aktywnych, a nie po liczbie scaleń. Zapisujemy historię zdarzeń `merge`/`split`, a przy pomiarze odtwarzamy ją w tej samej kolejności na każdym tekście, także przy wariancie bez usuwania. To narzędzie pomiaru. Nie przesądza docelowego encodera; alternatywą jest podział na najmniejszą liczbę tokenów (DP).

3. Powiązane prace

Pomysł powstał niezależnie, ale temat jest znany. **Picky BPE** [1] usuwa token w trakcie treningu, gdy stosunek liczności pary do liczności tokenu przekracza próg, np. 0,9; też przechowuje kolejność zdarzeń i dopuszcza powroty. **Scaffold-BPE** [2] oznacza tokeny o niskiej częstości jako „rusztowanie”. **LiteToken** [3] usuwa takie tokeny po fakcie z gotowego tokenizera. **Pruned BPE** [4] działa po treningu i przydziela zwolnione miejsca nowym tokenom, podając skrócenie tekstu o około 0,27–0,36% na tych samych danych. Nasz wariant różni się od [1]

głównie regułami decyzji: odniesieniem do n_0 oraz bilansem z f_{best}. Nie twierdzimy, że te reguły są nowe: przegląd był pobieżny i opierał się na fragmentach artykułów.

4. Metodyka

	nkjp1m	**DynaWord v0.2.6**
Dane	cały korpus, 18 381 dokumentów	26 źródeł, losowe dokumenty do limitu 20 mln tokenów `cl100k` na źródło; pominięte `ted_pl`
Liczba podziałów	**10**	**3**
Proporcja trening/test	75/25 według dokumentów; test to 24,9%–28,0% słów	75/25 według dokumentów, osobno w każdym źródle; test to 25,0%–26,5% słów
Jednostka podziału	tekst źródłowy (grupa fragmentów)	grupa w `nkjp1m` i `sejm_committee_transcripts`, dokument w 24 pozostałych
Wielkość treningu	696 tys.–726 tys. słów	85,1 mln–87,0 mln słów
Wielkość testu	240 tys.–271 tys. słów	29,0 mln–30,7 mln słów
Słowa niewidziane w teście	11,3%–12,1% wystąpień	1,9%–2,0% wystąpień
Alfabet	wszystkie znaki treningu (153–161)	znaki z co najmniej 1000 wystąpień w treningu (290–302)
Punkty pomiaru	61, od 200 do 30 000 tokenów	40, od 500 do 30 000 tokenów
Liczba treningów	10 × 3 warianty = 30	3 × 3 warianty = 9

Podział. Każdy podział to osobne losowanie z innym ziarnem (0–9 na nkjp1m, 0–2 na DynaWord). Losowo układamy grupy lub dokumenty i przydzielamy je do testu, aż obejmą 25% dokumentów, więc udział testu w słowach może się różnić. W DynaWord ziarno wyznacza też próbkę dokumentów z dużych źródeł, więc poszczególne podziały różnią się także danymi, a nie tylko przydziałem do zbiorów. To nie jest walidacja krzyżowa: zbiory testowe różnych podziałów mogą się nakładać.

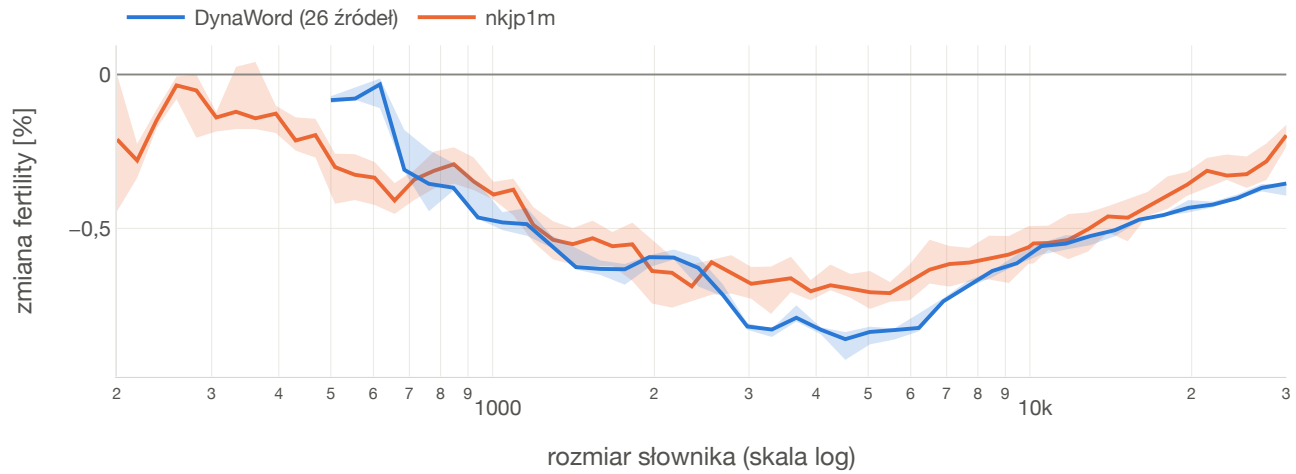
Trening i pomiar. Na każdym podziale trenujemy od zera wszystkie 3 warianty: klasyczne BPE, regułę 50% i regułę $L < f_{best}$. Dane testowe nie biorą udziału w treningu. Pre-tokenizacja działa jak w GPT-2 (`split_words`), a symbole startowe to znaki. Na DynaWord słowa ze znakami spoza alfabetu nie biorą udziału w treningu, a w teście taki znak liczy się jako jeden token. Rozmiar słownika obejmuje alfabet.

Metryka. Fertility to średnia liczba tokenów na kawałek zawierający litery, ważona liczbą wystąpień; interpunkcja i same liczby nie wchodzą do średniej. Słowo testu jest „widziane”, jeśli wystąpiło w treningu. Na DynaWord mierzymy też każde źródło osobno, na jego części testowej.

Analiza. Warianty porównujemy parami, na tym samym podziale: zmiana to $100 \cdot (F_{wariant}/F_{BPE} - 1)$. Pozwala to oddzielić efekt od różnic między podziałami, które są tego samego rzędu: fertility klasycznego BPE na teście przy 30 000 tokenów różni się między podziałami o 1,4% na nkjp1m i 0,5% na DynaWord, a efekt to odpowiednio 0,20% i 0,35%. Podajemy medianę oraz najmniejszą i największą wartość, czyli rozpiętość, a nie przedział ufności. Przy tak małej liczbie podziałów, zwłaszcza 3, nie liczymy testów istotności. Punkty pomiaru jednego biegu nie są niezależne, bo każdy słownik powstaje z poprzedniego.

Kontrola poprawności. Przykład z sekcji 1 daje wyniki policzone ręcznie, a 300 losowych małych korpusów potwierdza zachowanie tekstu i zgodność odtworzenia z treningiem. Wariant bez usuwania daje na nkjp1m te same scalenia co dotychczasowa implementacja, a podział 0 odtwarza wcześniejsze liczby. Implementacja: Rust (PyO3) z indeksem „para → słowa”.

5. Wyniki



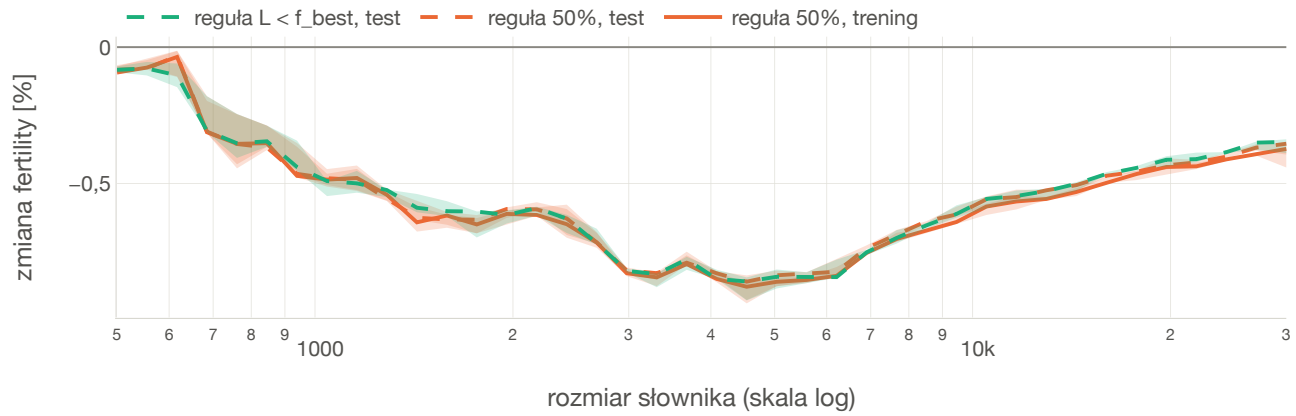
Rys. 1. Zmiana fertility na teście po regule 50% względem klasycznego BPE. Linia: mediana, pasmo: najmniejsza i największa wartość (10 podziałów nkjp1m, 3 podziały DynaWord). Poniżej zera wariant daje mniej tokenów.

Słownik	Fertility testu, BPE	Fertility testu, 50%	Test, 50%	Test, $L<f_{best}$	Trening, 50%	Niewidziane, 50%	nkjp1m: najbliższy słownik, test, 50%
1 043	2,852	2,838	−0,48% (−0,51 do −0,45)	−0,49% (−0,55 do −0,49)	−0,49% (−0,51 do −0,46)	+0,19% (−0,13 do +0,20)	1 004: −0,39% (−0,41 do −0,35)
2 979	2,178	2,160	−0,82% (−0,83 do −0,81)	−0,82% (−0,83 do −0,81)	−0,83% (−0,84 do −0,83)	−0,43% (−0,44 do −0,37)	3 029: −0,68% (−0,73 do −0,63)
4 534	1,973	1,955	−0,86% (−0,93 do −0,84)	−0,86% (−0,93 do −0,84)	−0,88% (−0,94 do −0,86)	−0,38% (−0,47 do −0,36)	4 631: −0,70% (−0,73 do −0,65)
10 500	1,646	1,637	−0,56% (−0,59 do −0,55)	−0,56% (−0,59 do −0,55)	−0,58% (−0,62 do −0,57)	−0,28% (−0,36 do −0,19)	10 827: −0,55% (−0,57 do −0,49)
30 000	1,373	1,368	−0,35% (−0,39 do −0,35)	−0,35% (−0,39 do −0,34)	−0,37% (−0,44 do −0,37)	−0,04% (−0,06 do +0,01)	30 000: −0,20% (−0,23 do −0,16)

Tabela 1. DynaWord i nkjp1m, mediana zmiany względem klasycznego BPE; w nawiasie rozpiętość między podziałami.

Główne obserwacje

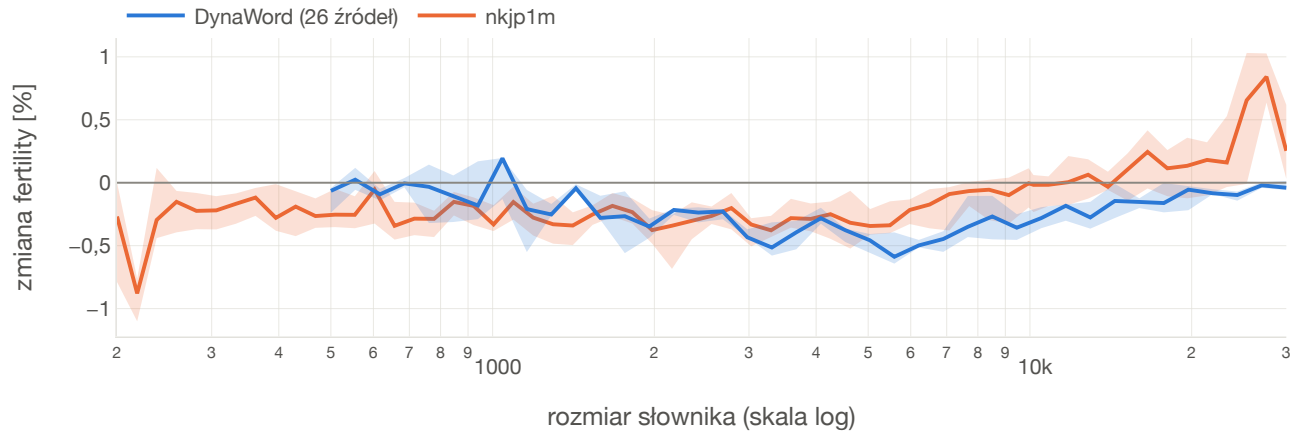
- **Powtarzalność.** Na DynaWord test poprawia się na wszystkich 3 podziałach w 40 z 40 punktów; na nkjp1m na wszystkich 10 podziałach w 58 z 61 punktów, a wyjątki występują tylko poniżej 400 tokenów.
- **Kształt efektu.** Zysk rośnie do płaskiego maksimum między około 3000 a 6000 tokenów, a potem maleje: na DynaWord do −0,35%, na nkjp1m do −0,20% przy −0,71% w okolicy 5 489 tokenów.
- **Większe dane zmniejszają rozjazd trening–test.** Przy 30 000 tokenów na DynaWord zysk na teście i na treningu jest prawie równy (−0,35% i −0,37%), a na nkjp1m wyraźnie się rozchodzi (−0,20% i −0,56%). Część tej różnicy może wynikać ze słabszego podziału po dokumentach (sekcja 6).



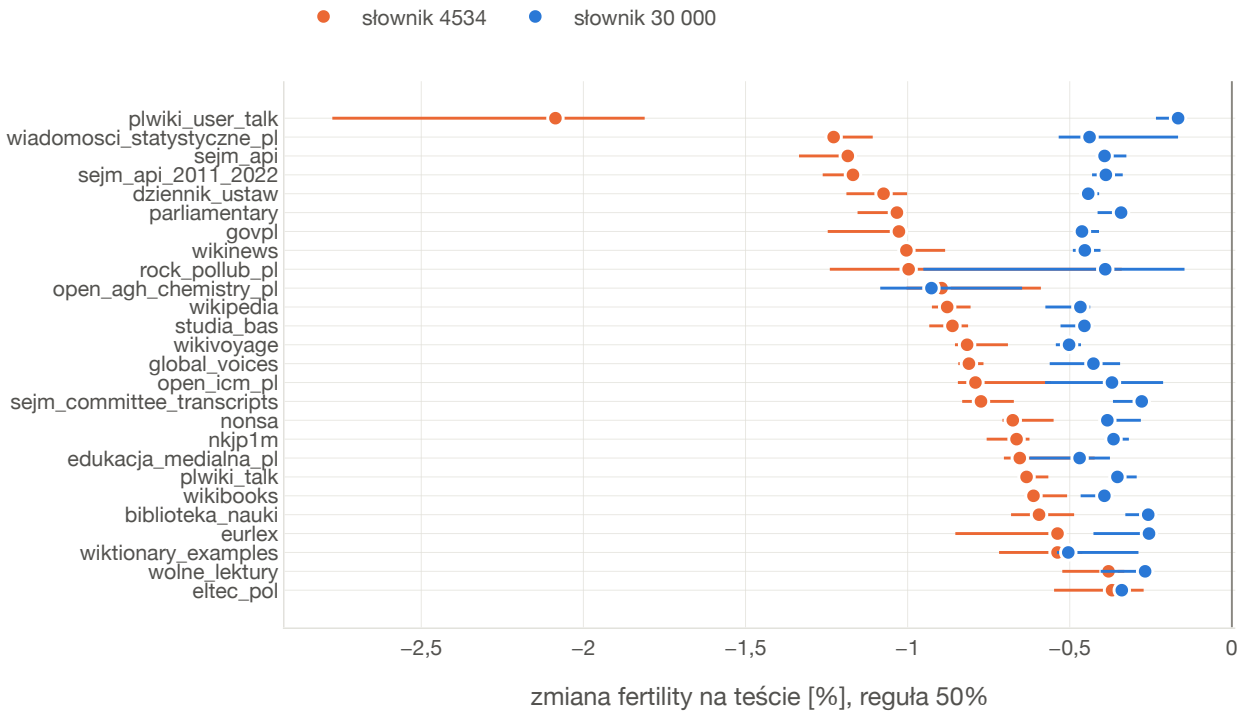
Rys. 2. DynaWord: porównanie reguł i rozjazd trening–test. Reguły są nierozróżnialne w granicach rozrzutu.

Wariant (mediana z 3 podziałów DynaWord)	Scalenia	Usunięcia	Powroty	Usunięte na stałe
bez poprawki	29 709	0	0	0
reguła 50%	39 404	9 682	7 761	1 919
reguła L < f_best	33 562	3 847	2 156	1 691

Tabela 2. Praca algorytmu przy słowniku 30 000. Reguła 50% ma więcej usunięć, z których większość kończy się powrotem, a liczba tokenów usuniętych na stałe jest podobna w obu regułach.



Rys. 3. Słowa niewidziane w treningu. Na nkjp1m przy dużym słowniku wynik się pogarsza (+0,25% przy 30 000), na DynaWord jest bliski zera (−0,04%), ale takie słowa to tam tylko około 2% testu.



Rys. 4. Zmiana na teście według źródeł, DynaWord, reguła 50%. Wąs: najmniejsza i największa wartość z 3 podziałów. Najmniejsze źródła mają mało danych testowych i dużą rozpiętość. Przy 4534 tokenach `plwiki_user_talk` zyskuje najwięcej (–2,09%), ale przy 30 000 ma najmniejszy zysk (–0,17%). Mediana każdego źródła przy 30 000 jest poniżej zera (najsłabsze: –0,17%). Przyczyny różnic nie badaliśmy.

6. Ograniczenia

- **Podział po dokumentach w 24 źródłach.** Ich ID nie pozwalają pewnie grupować fragmentów tego samego tekstu. Tam, gdzie takie fragmenty istnieją, test jest zbyt łatwy, co może zaniżać udział słów niewidzianych i rozjazd trening–test. Autorzy datasetu wskazują też częściowe duplikaty między `wolne_lektury` i `eltec_pol`.
- **Mała liczba podziałów DynaWord.** 3 podziały pokazują kierunek, ale słabo oceniają rozrzut. Podziały nie są też niezależnymi próbami, bo pochodzą z tego samego korpusu.
- **Metryka.** Fertility nie mówi o jakości modelu ani o rzadkich tokenach, które [1] wskazuje jako główną korzyść. Zmiana 0,35–0,86% jest niewielka w liczbach bezwzględnych.
- **Encoder.** Pomiar odtwarza historię zdarzeń, więc wynik może się zmienić przy encoderze DP lub longest-match.
- **Wybrane parametry.** Jeden próg dla znaków (1000), jeden limit na źródło i wariant „słowa + znaki”. Na bajtach tego nie mierzyliśmy.
- **Czas.** Jeden pełny pomiar 3 wariantów na DynaWord trwał 12–19 min. Ten sam trening bez usuwania miał rozrzut czasu około 3-krotny, więc czasy porównawcze nie są wiarygodne. Nie podajemy zysku czasowego.

7. Wnioski i dalsze kroki

Usuwanie tokenów pośrednich poprawia fertility w sposób mały, ale powtarzalny, także na teście i na wszystkich badanych typach tekstu. Prosty próg 50% działa równie dobrze jak reguła z bilansu kosztów, a w praktyce można preferować $L < f_{\text{best}}$ ze względu na mniejszą liczbę usunięć i prostszą historię zdarzeń.

Kolejne kroki: (1) krzywa z encoderem DP na tych samych słownikach, (2) analiza, które tokeny są usuwane i skąd szczyt przy kilku tysiącach tokenów, (3) powtórzenie z podziałem po tekstach tam, gdzie to możliwe, (4) sprawdzenie wpływu na trening małego modelu językowego, bo to on ostatecznie rozstrzyga o wartości tej zmiany.

Literatura

[1] Chizhov i in., *BPE Gets Picky: Efficient Vocabulary Refinement During Tokenizer Training*, EMNLP 2024. arxiv.org/abs/2409.04599
[2] Lian i in., *Scaffold-BPE: Enhancing Byte Pair Encoding for Large Language Models with Simple and Effective Scaffold Token Removal*, AAAI 2025. doi.org/10.1609/aaai.v39i23.34633
[3] *LiteToken: Removing Intermediate Merge Residues From BPE Tokenizers*, 2026. arxiv.org/abs/2602.04706
[4] *Pruned BPE: Post-training Visibility Pruning and Token Reallocation for Byte Pair Encoding*, 2026. arxiv.org/abs/2608.00837
Dane: SlayerLab/polish-dynaword, wersja v0.2.6. Kod: `rust/bpe/src/lib.rs`, `experiments/tokenizer/09`–`17`. Kod i analiza powstały z pomocą asystenta AI (Claude); pomysł i decyzje eksperymentalne: autor.